CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# An introduction to Git for individuals

CMT501

Module leader: Carl Jones

# Managing yourself to play well in a team

- Applying control to your work
- Recording your thoughts as you go
- Enabling options
- Having a backup
- Enabling feedback loops

# Interactive learning

- Bit of talk
- Bit of watching
- Bit of doing

# What's wrong with my local files?

**Positive**

- Fast access
- No server required
- Tools work well with it

**Negative**

- No backup
- No easy history
- No easy linkage
- No easy merge

# What's wrong with OneDrive*?

**Positive**

- Fast access
- No server required until sync
- History
- Backup

**Negative**

- No easy linkage
- No easy merge
- Tools don't always work well with it

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Version Control Systems

- Many out there
  - Subversion (dominant in 1990/2000s)
  - Git (currently dominant)
  - Roll your own (Google)

- We are going to use git.

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Git




- Released in 2005
- Created by Linus Torvalds
- Open-source
- Used by 95% of developers (2022 Survey)

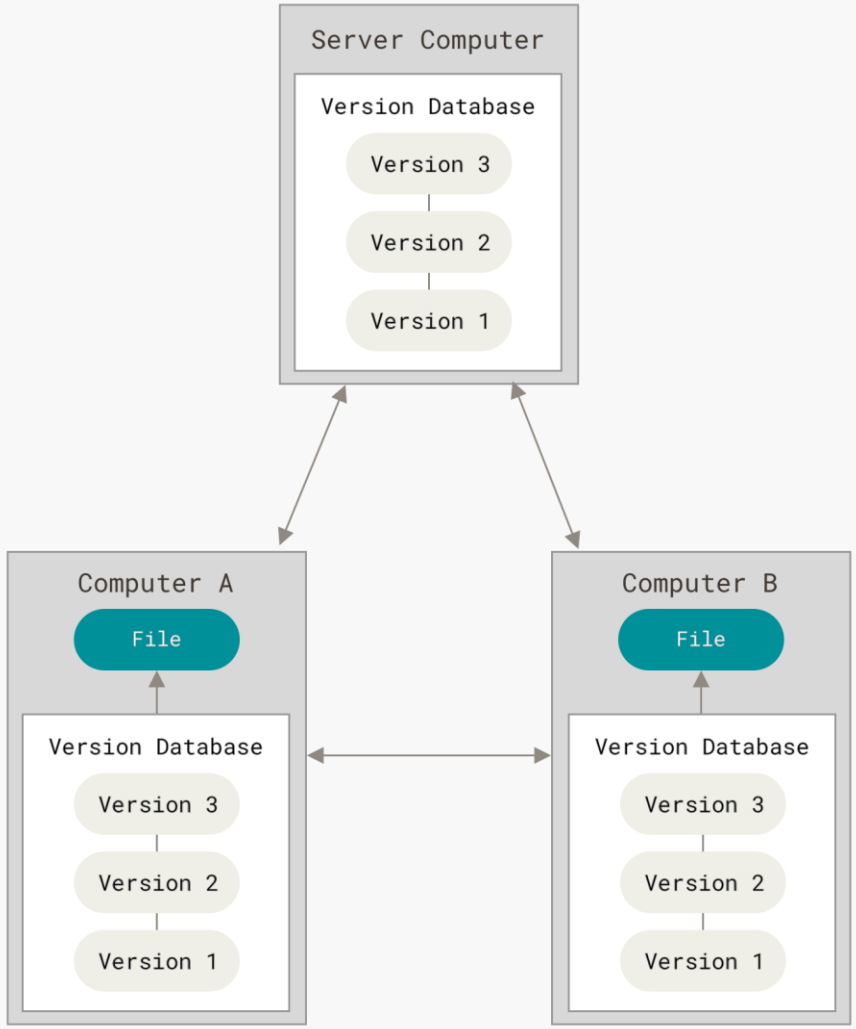


https://upload.wikimedia.org/wikipedia/commons/thumb/0/01/LinuxCon_Europe_Linus_Torvalds_03_%28cropped%29.jpg/500px-LinuxCon_Europe_Linus_Torvalds_03_%28cropped%29.jpg

# Git - a distributed version control system

- Works on the notion of a ‘repository’
  - A set of files with associated metadata in a “.git” directory
- A local repository and a remote repository for backup…
  - …and (spoiler) for collaboration.
- Supports local work with full tracking
  - No need to be constantly connected to a server

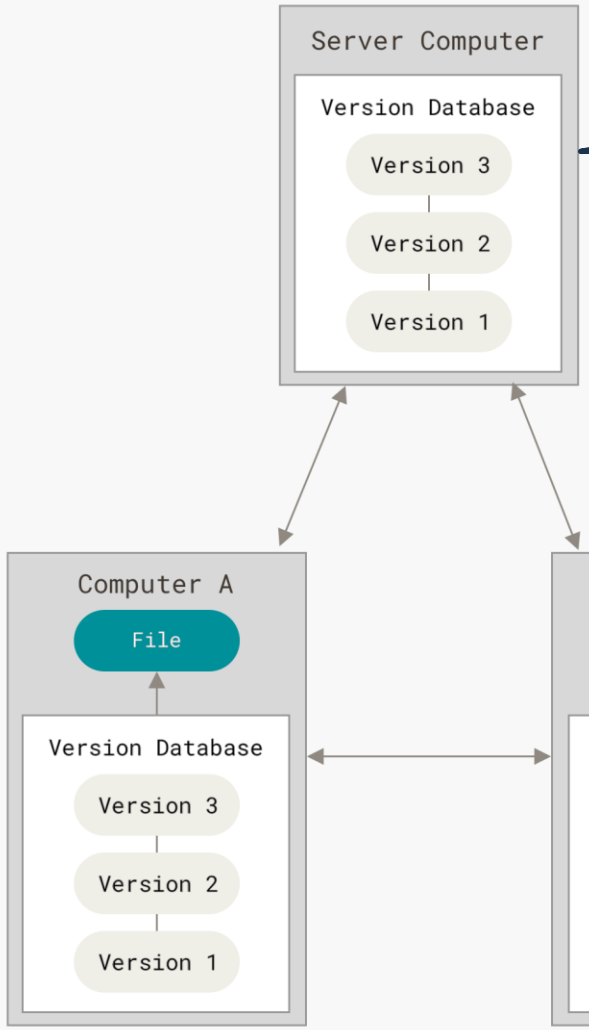
# Distributed Version Control




The version control system runs on each computer and changes are “merged”.

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

# In our domain




This will be
git.cardiff.ac.uk

(running Gitlab)

This will be your own device(s) running Git.

https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Where to start

https://git-scm.com/

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Install Git (if not already installed)

- Pick the installer for your platform
- Standard options work well
- On Windows, choose “git bash” option.
  - You can install for Windows command prompt/terminal as well

# Tell git about yourself

- You want git to know about you so that changes are corrected annotated.
- To do this, we will use "`git config`" and the "`global`" option

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Git Config

```
git config –global user.name “Bugs Bunny”

git config –global user.email “bugs@toontown.com”
```

# Create a folder for a repository

- To create a git repository, we need to pick a directory as the root.

- ADVICE: do not use a OneDrive directory.
- ADVICE: use `C:\users\<you>\code\` …

- e.g `C:\users\<you>\code\tutorials\git-for-one`

- Do this using Windows Explorer or command line

# Open Git Bash in directory

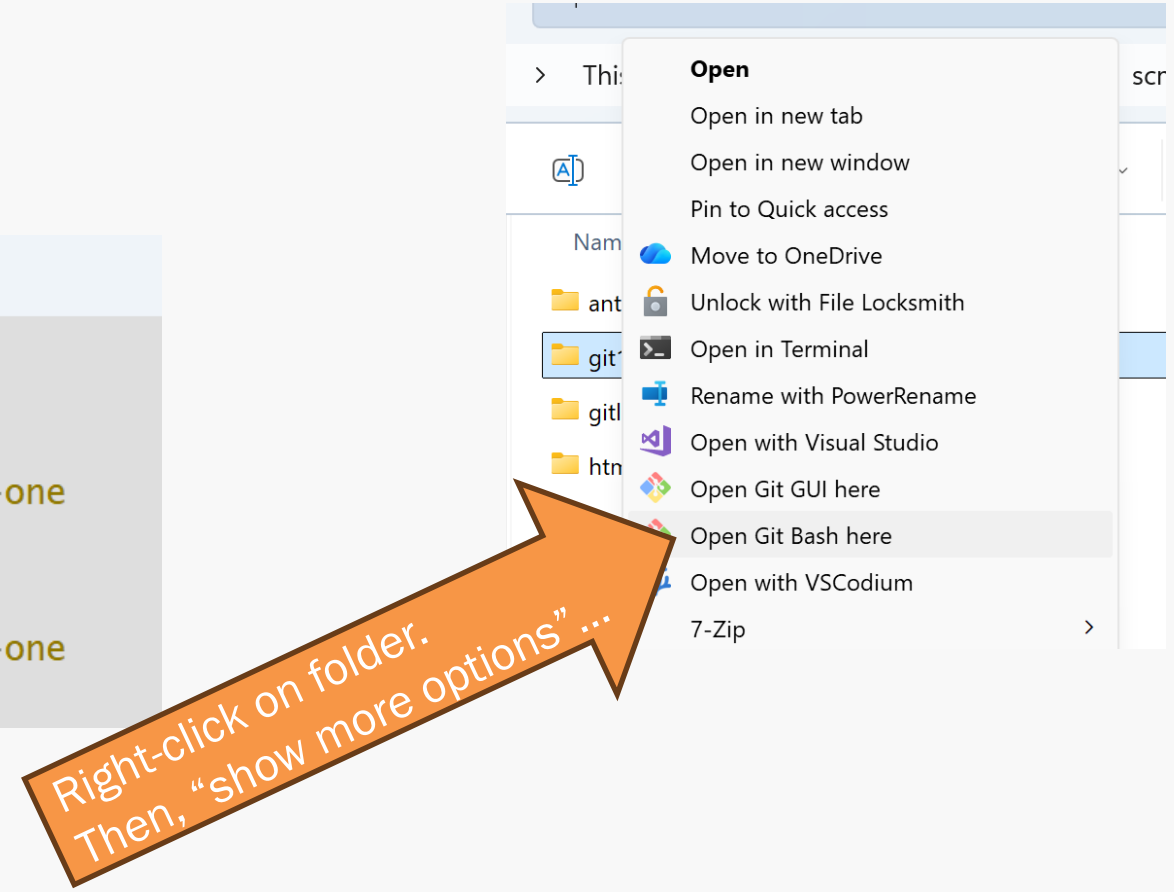
- We need to open git bash and navigate to the directory that we just created.

```
MINGW64:/c/Users/scmcj1/code/demos/git-for-one

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos
$ cd git-for-one/

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one
$ pwd
/c/Users/scmcj1/code/demos/git-for-one

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one
$
```



# Create Git repository

- The command to create a git repository is "`git init`".
- Before running the command, run "`ls -a`".
- Note the contents of the directory.
- Now run "`git init`".
- Now look at the contents of the directory.
  - What's different?

# What has happened?

- Run "`git status`".




- If your branch is called "master"...
  - Run "`git config --global init.defaultBranch main`"
  - Delete the "`.git`" directory
  - Run "`git init`" again.

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# What is this ".git" folder for?

- It is the metadata for your repository.
- Essentially, it records the audit trail of changes.
- Also knows the configuration of your repository.

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Let's create some content

- Create a HTML file called "`index.html`" and populate with some "hello world" content.
- Save the file and check the contents of your directory.
- Check the status of your repository

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# What's this "untracked" thing?



```
ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ ls -a
./  ../  .git/  index.html

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        index.html

nothing added to commit but untracked files present (use "git add" to track)
```

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Untracked

- We have a file, but git doesn't care about it yet.
- We need to tell git to start tracking our file and move it to our "staging" area.
- Git needs to be told. Would you want to put generated binaries into version control by default?

- To tell git to track a file, use "`git add <file>`"
- Normally, we add all files with "`git add .`"

# Now we're tracked



```
ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git add .

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Your turn to track...

# What's this branch thing?



```
ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git add .

ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   index.html
```

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# A train of thought

- Think of each branch as a train of thought or a timeline.
- Branches can be associated with a task or feature.
- The "`main`" branch (formerly "`master`" branch) is default.
- We will learn how to create and use branches later.

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# When happy, we commit

- Let's assume that we're happy with our work.
- At these points, we want to "commit" our work to the repository
- To commit, we use the "`git commit`" command.
- For a commit, we need to provide a commit message.
- The message should be useful to those who will read it in future. It should discuss the changes, their motivations and the thought process behind them.

# Git commit with message

- `git commit –m “Initial home page. Just need to say hello”`

- If you leave out the “`–m`”, an editor will open for the message.
- Your IDE will often provide a text box for the message.
- There are numerous standards out there for commit messages.

# Check the status again

- Everything is committed.
- No changes outstanding.
- Run "`git log`"
- Your commit should appear.

```
$ git status
On branch main
nothing to commit, working tree clean
```

```
ID+scmcj1@WR3OE3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git log
commit 9c60ef5023a621b396a8c34089f4b3ce6a8f8e69 (HEAD -> main)
Author: Carl Jones <jonesc162@cardiff.ac.uk>
Date:   Mon Jun 15 10:51:06 2026 +0100

    Initial home page. Just Hello World for now.
```

# Repeat the process

- Make a series of changes.
- Play with the commands.
- Add, then commit.
- Add, change, then commit.
- Add, change, commit, change, commit
- Keep checking the audit trail.

# Everything is currently local

- Lose your laptop or even  just your drive and you've lost everything.
- We could just backup the folder to another file store.
- Git allows us to synchronise changes between repositories.

# Hosted repositories

- There are a number of players in the hosted repository market.
- GitHub (owned by Microsoft)
- GitLab
- BitBucket (from Atlassian)

# GitHub

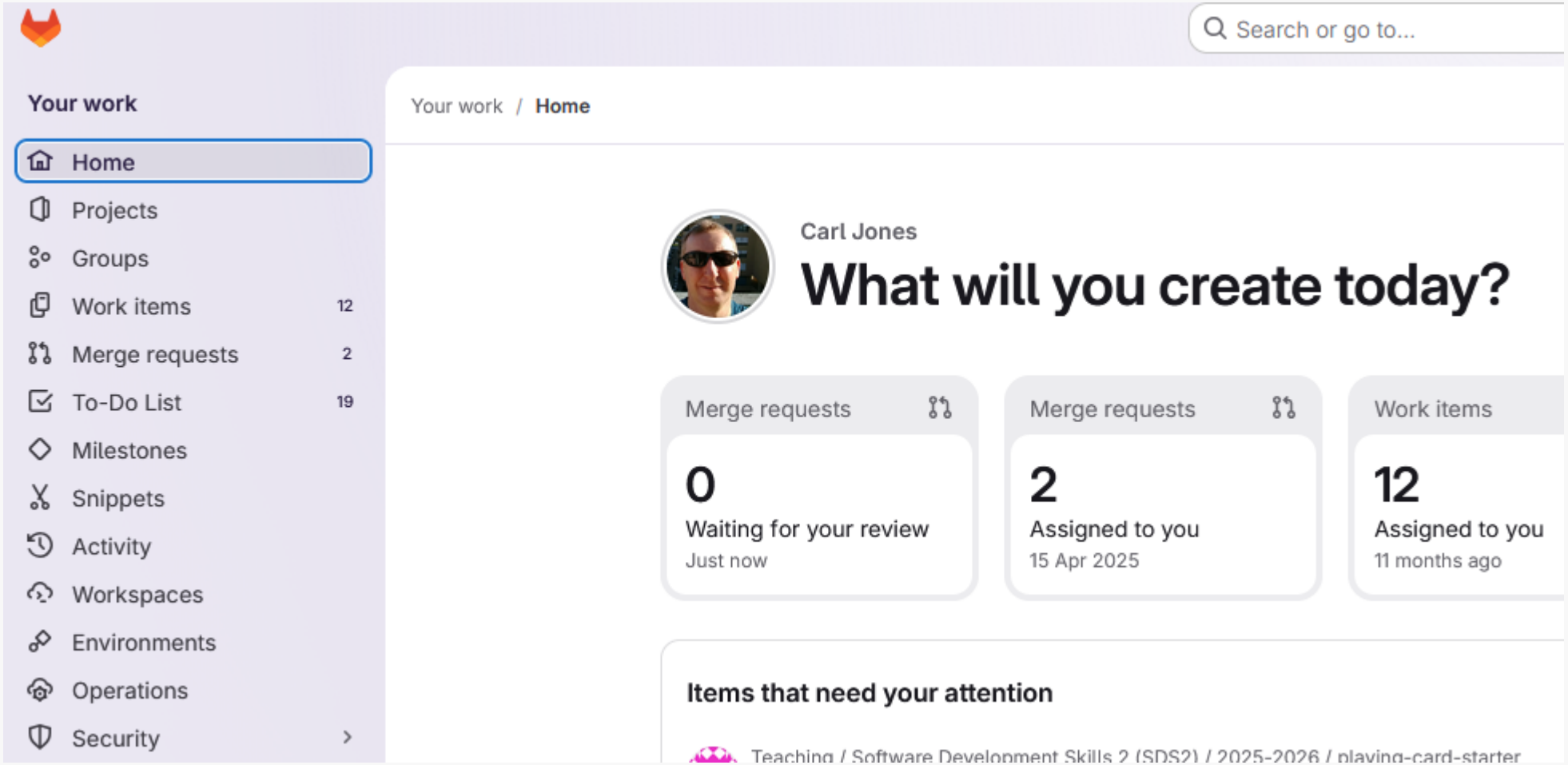
- The market leader.
- It is well worth you having a GitHub account.
- It can be your public portfolio.
- GitHub offers education packs
  - A little bit of Co-Pilot (coding agent)

# Gitlab

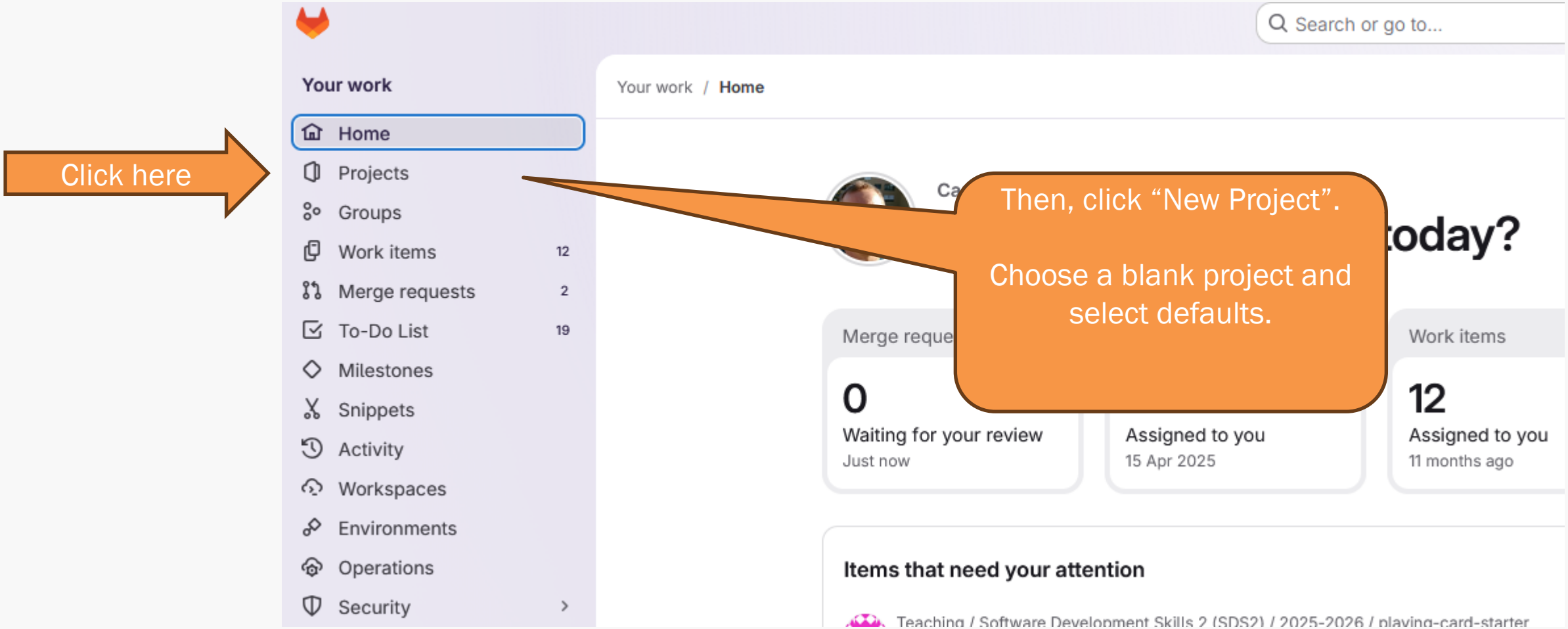
- Offers a full application life cycle management (ALM) tool around git.
- Not as big a player in education.
- Self-hosted by Cardiff University.

- Go to "`https://git.cardiff.ac.uk`"

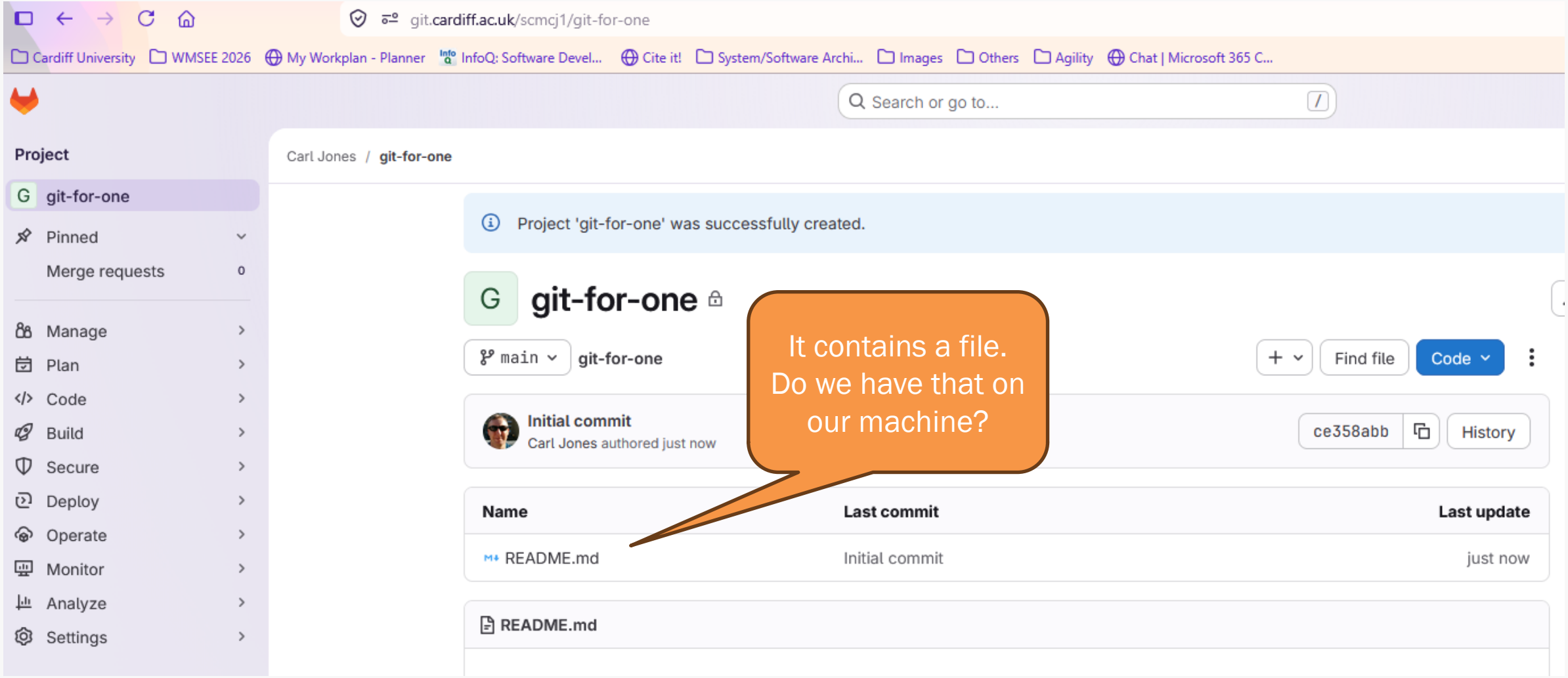
# Gitlab Home Page



# Gitlab Projects



# Your Gitlab repository

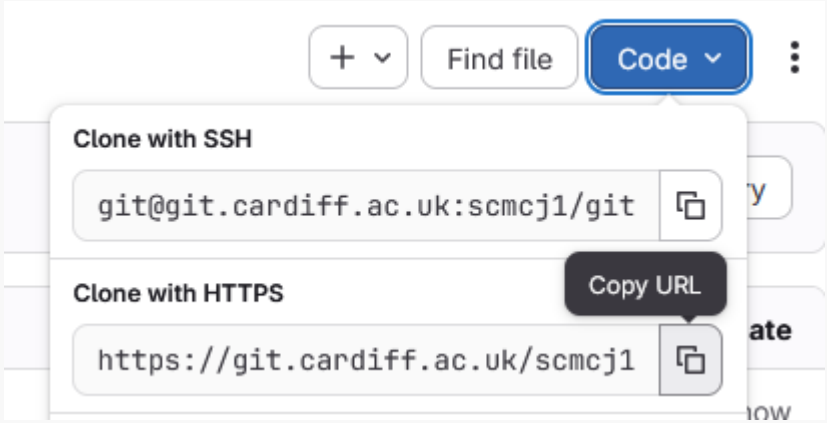


# Two repositories

- We now have two repositories; one local and one on Gitlab.
- They contain files that the other doesn't.
- We need to connect them and synchronise them.
- We will then make changes locally and synchronise to Gitlab.

# Link the repositories

- Click “Code” and select the “https” URL.




- In git bash, run…
- `git remote add origin <the url>`

# Synchronise the repositories

- We have a choice; do we send the local changes to Gitlab, or the Gitlab changes to our local repository.
- This is known as “push” or “pull” respectively.
- Let’s try to push.
- `git push origin main`

# What was the order of change?

- The remote was created after our local and therefore the commit of "`README.MD`" was after our commit. Therefore, the most up to date version is the remote and we need that in our local repository. Let's pull.
- `git pull origin main`

```
ID+scmcj1@WR30E3A4841E62 MINGW64 ~/code/demos/git-for-one (main)
$ git push origin main
To https://git.cardiff.ac.uk/scmcj1/git-for-one.git
 ! [rejected]        main -> main (non-fast-forward)
error: failed to push some refs to 'https://git.cardiff.ac.uk/scmcj1/git-for-one
.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
```

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Unrelated histories?

- This might fail due to “unrelated histories”.
- The two repositories were created independently.
- Use “`git pull origin main –allow-unrelated-histories`”
- An editor will appear for the commit message.
  - Accept the default (type “`:wq`” to exit the standard Windows editor)
- We’ll talk about how to avoid this problem later.

# Check the content

- Check the contents of your local repository.
- Are there now two files?

- Let's sync back to Gitlab
- `git push`

- Check Gitlab

# Explore push and pull

- Change files locally and push the changes to Gitlab.
- Once histories are related, do you have any issues pushing?
- When do you think issues might occur?

# What if we started with the backup?

- Let's imagine that we started by creating the project on Gitlab.
- Either…
  - Create another empty folder (call it "git-for-one-B" or similar)
  - Delete all files from existing folder (including the ".git" folder).
- Get the https URL of the gitlab repository.
- Run "`git clone <the URL>`"

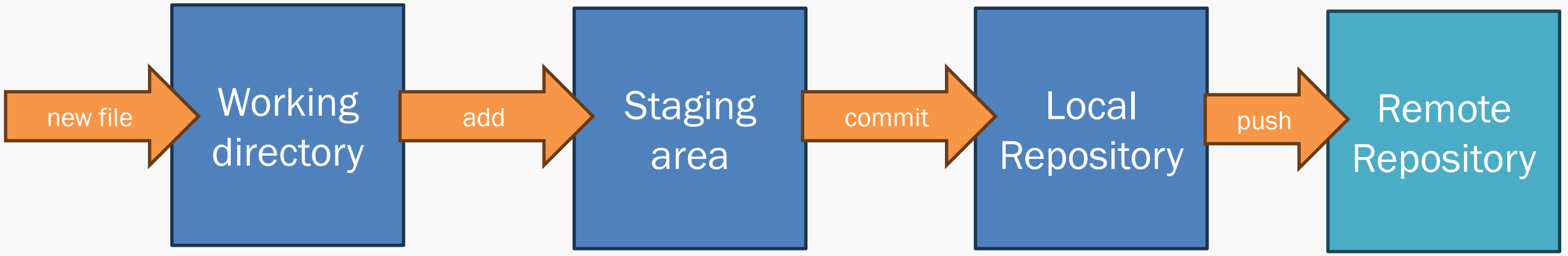
# The repository is copied

- You should have a new folder in your folder.
- It will be named after your Gitlab repository.
- The sync is now automatic. The remote is setup.
- Try add, commit, push – it should just work.

# Recap

- What we can now do:
  - Create local files
  - Tell to track them
  - Commit changes with comments
  - Synchronise with a remote repository
  - Check the status of git
  - Check the history of git

# Git flow transitions



# Branches

- So far we've only used the "main" (or "master") branch.
- For some workflows, this may be more than enough.

- One of git's advantages over other version control systems is that branching is (computationally) cheap.

# Why would we need branches?

- SPOILER: they are more useful in a team scenario

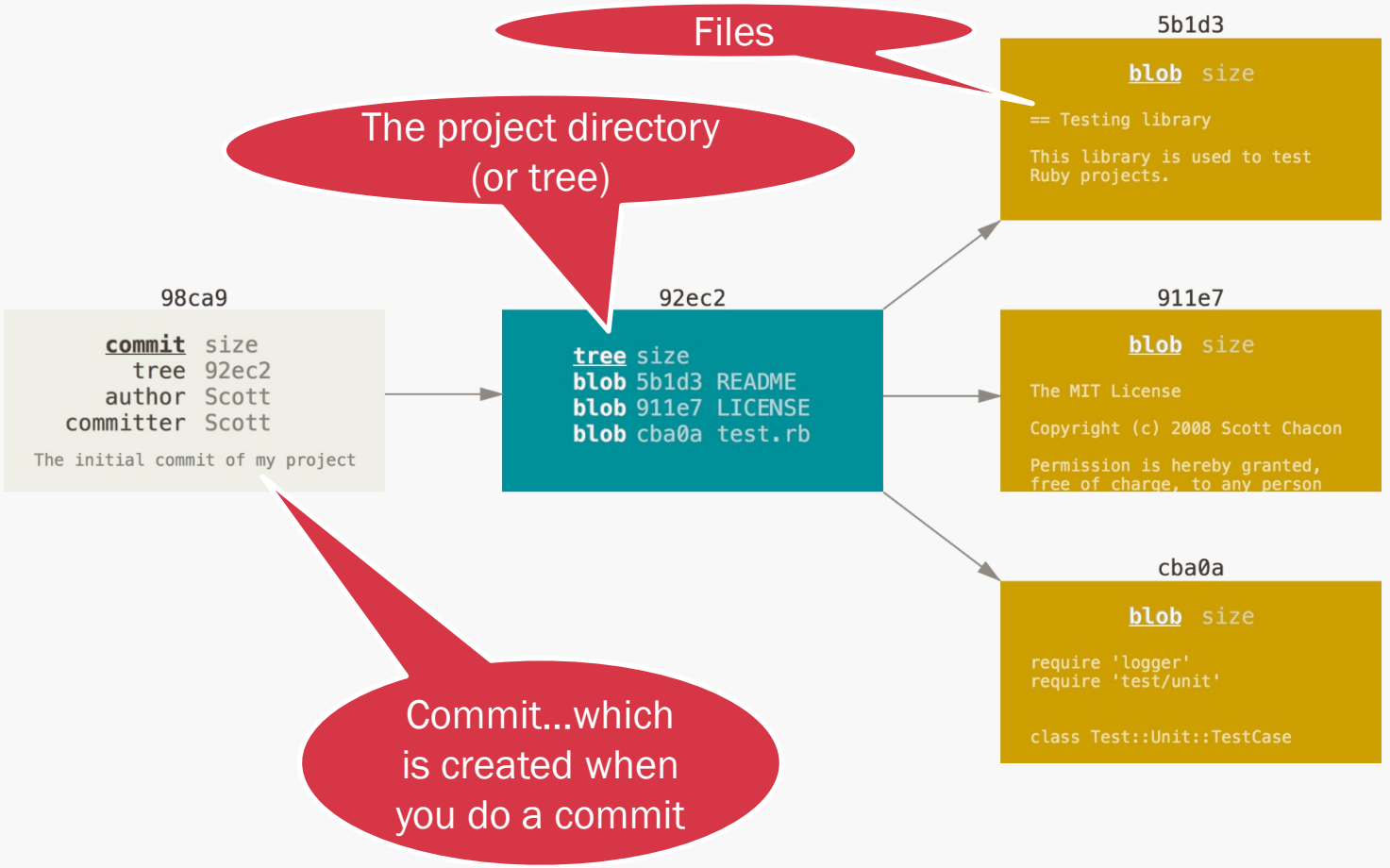
- What if:
  - You want to explore more than one option to complete a task?
  - Every commit to the remote repository caused the code to go live?
    - Partial work might need to be held back.
  - You're using an AI agent and you want to review the code before it gets to "main"?

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# How commits work?

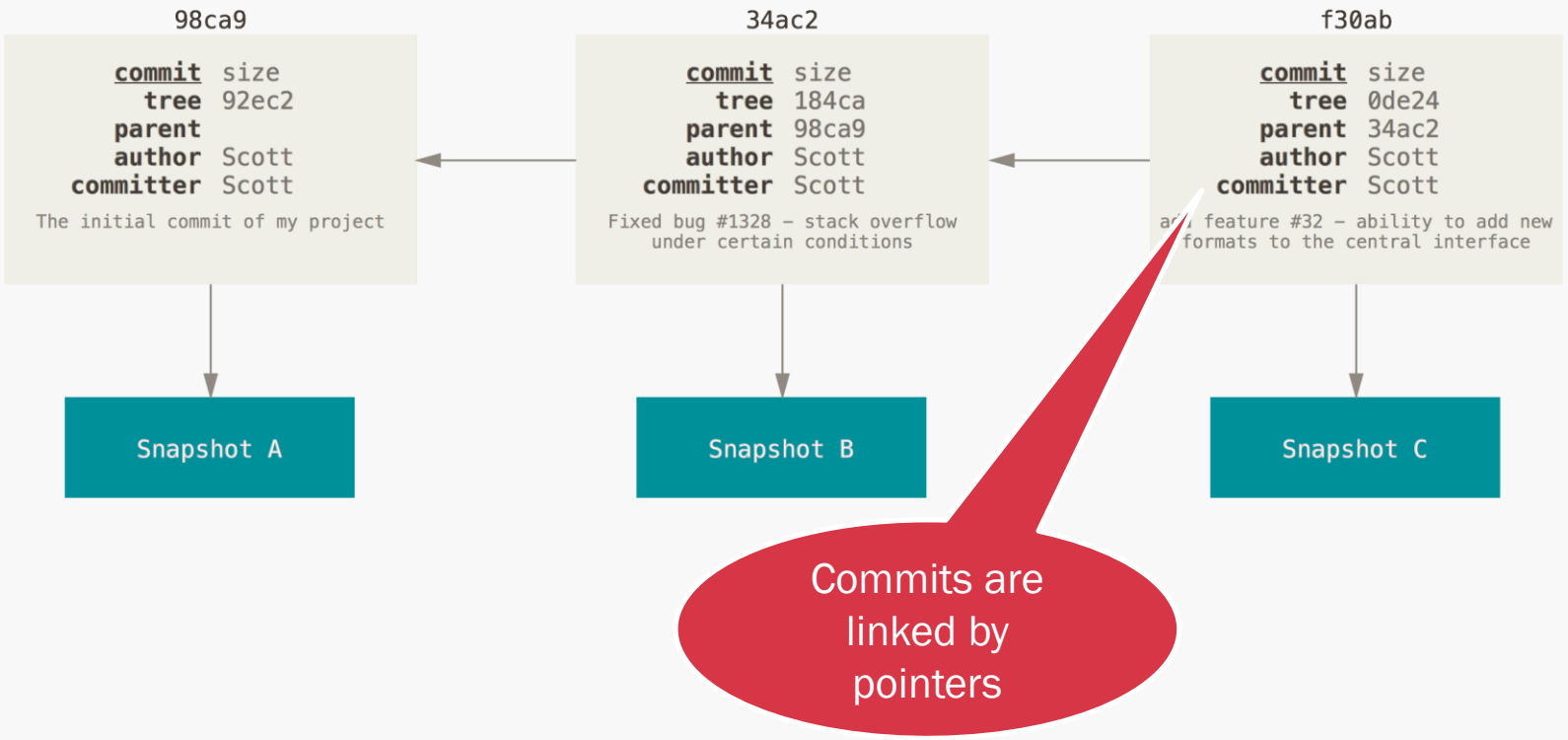
- To understand how branches work in Git, we need to understand a bit more about how commits work in Git.

# Commits in Git



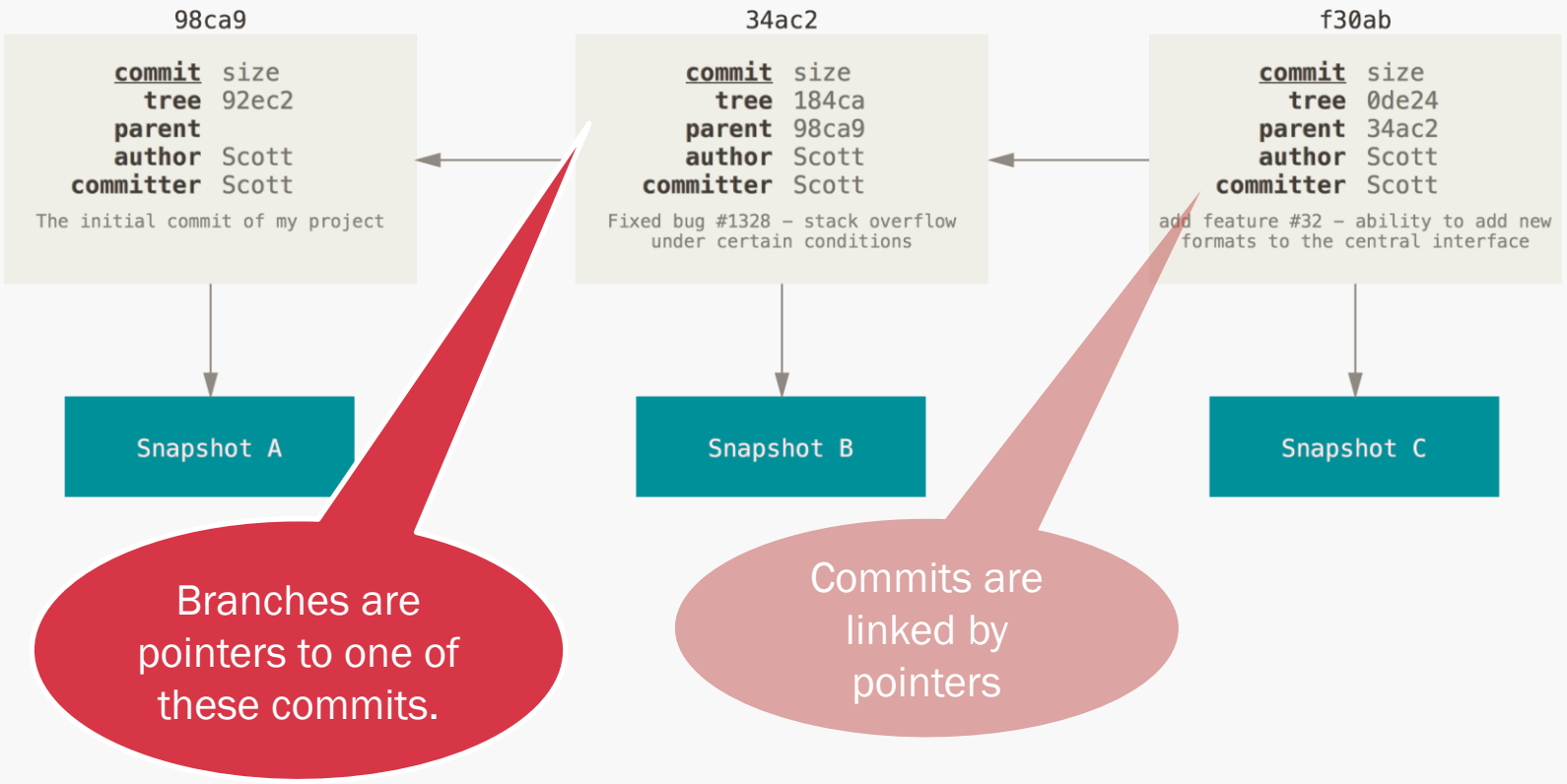

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Branches in Git



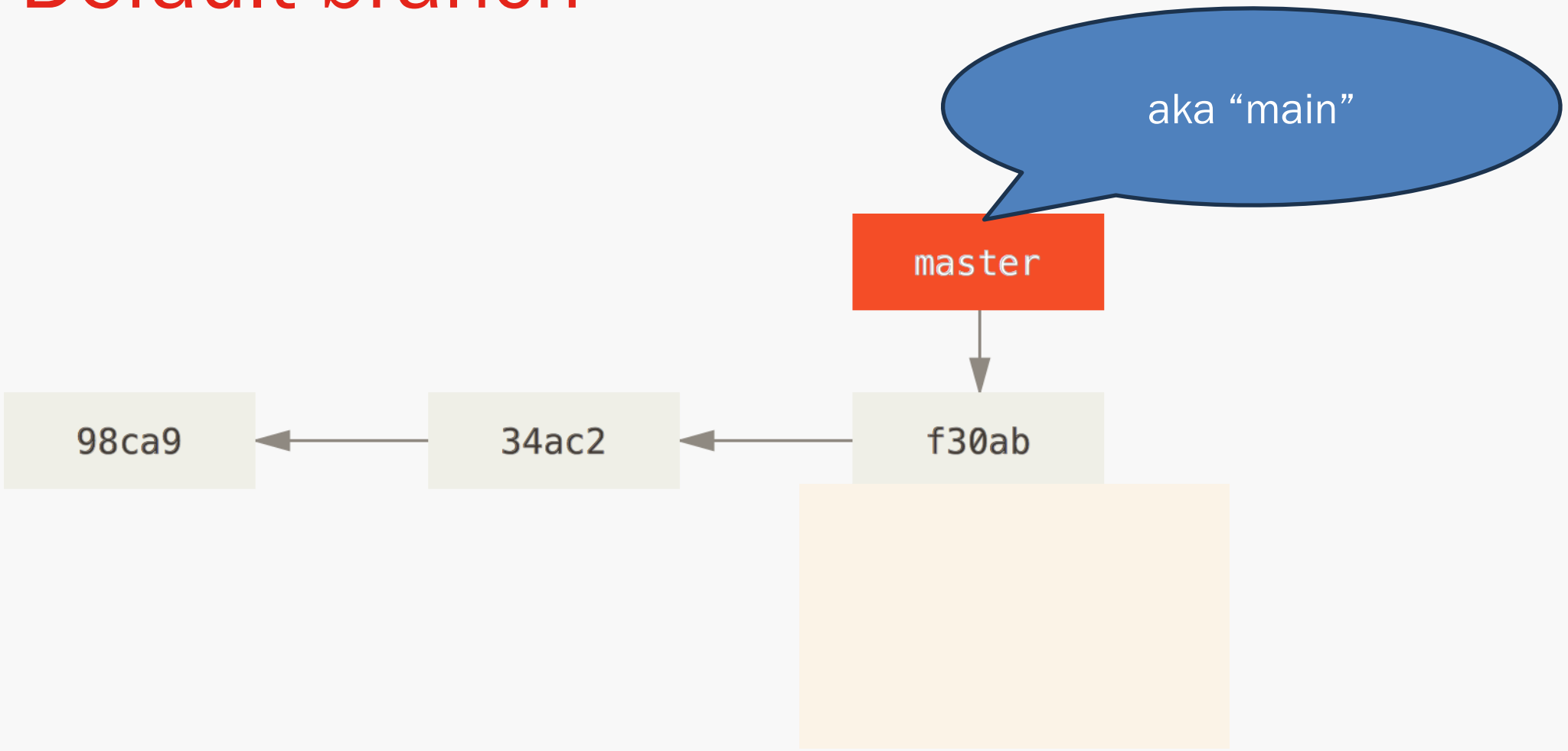

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Branches in Git



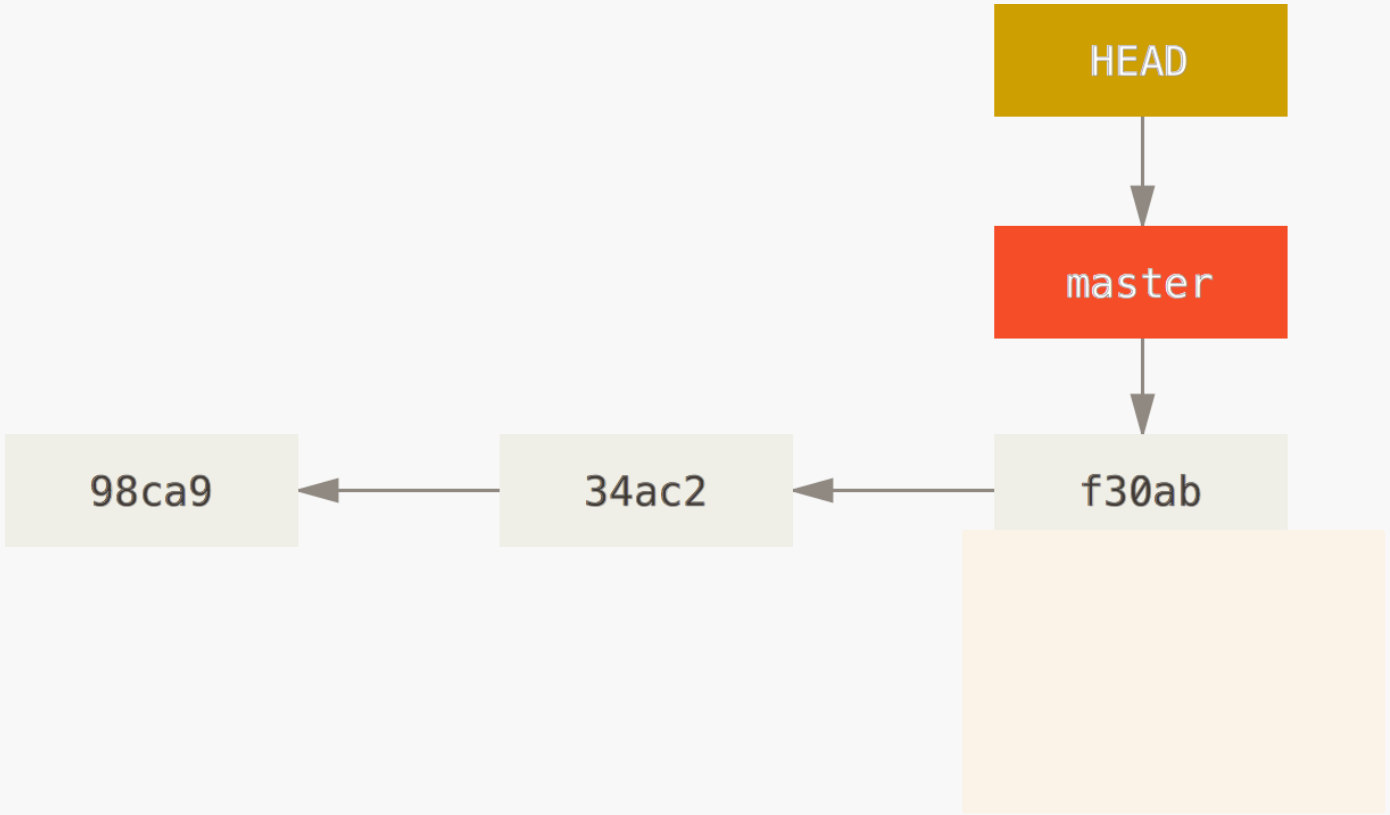

https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Default branch




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

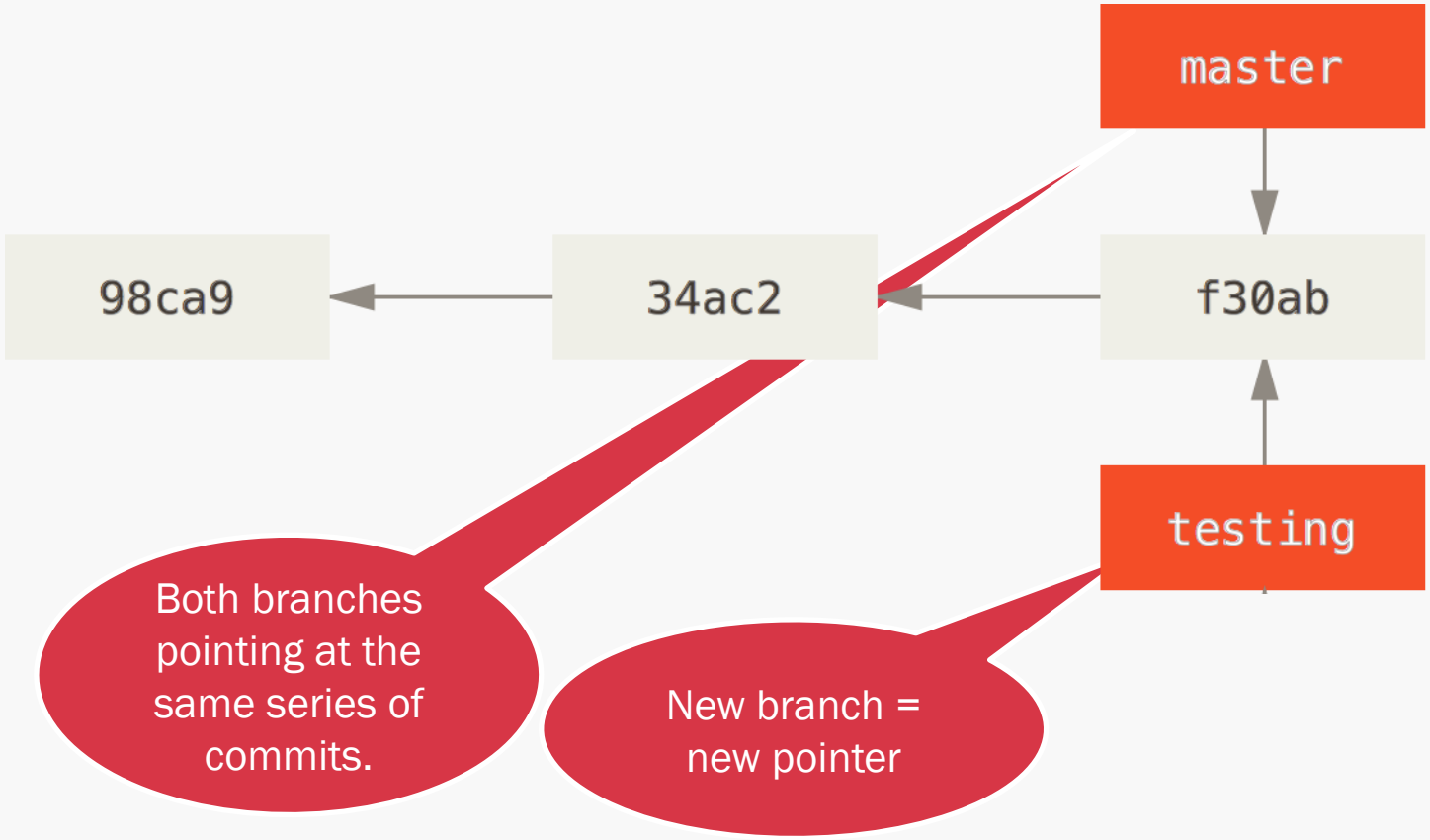
# HEAD tells us the current branch




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Create a new branch

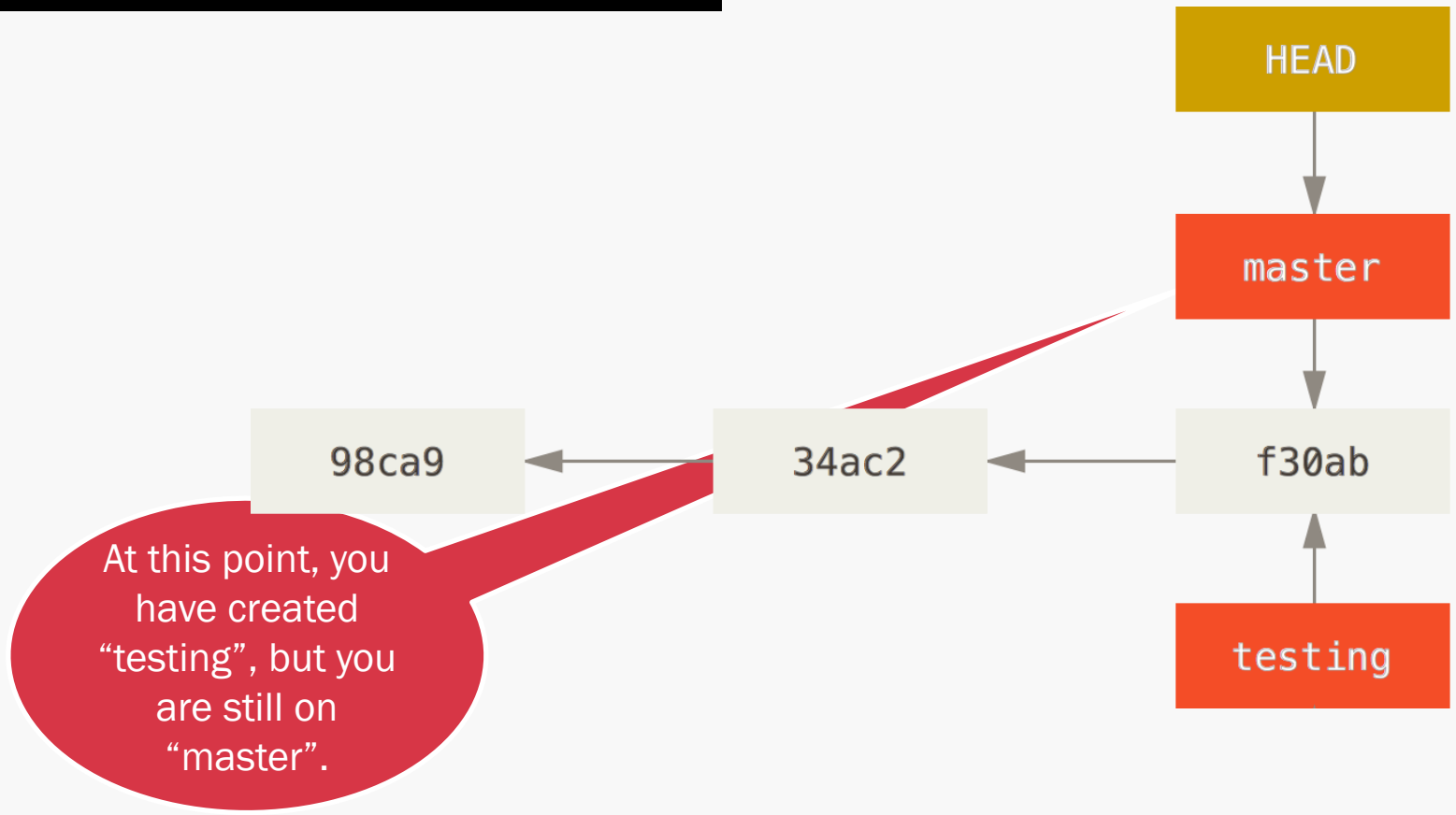
```
$ git branch testing
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# HEAD tells us the current branch

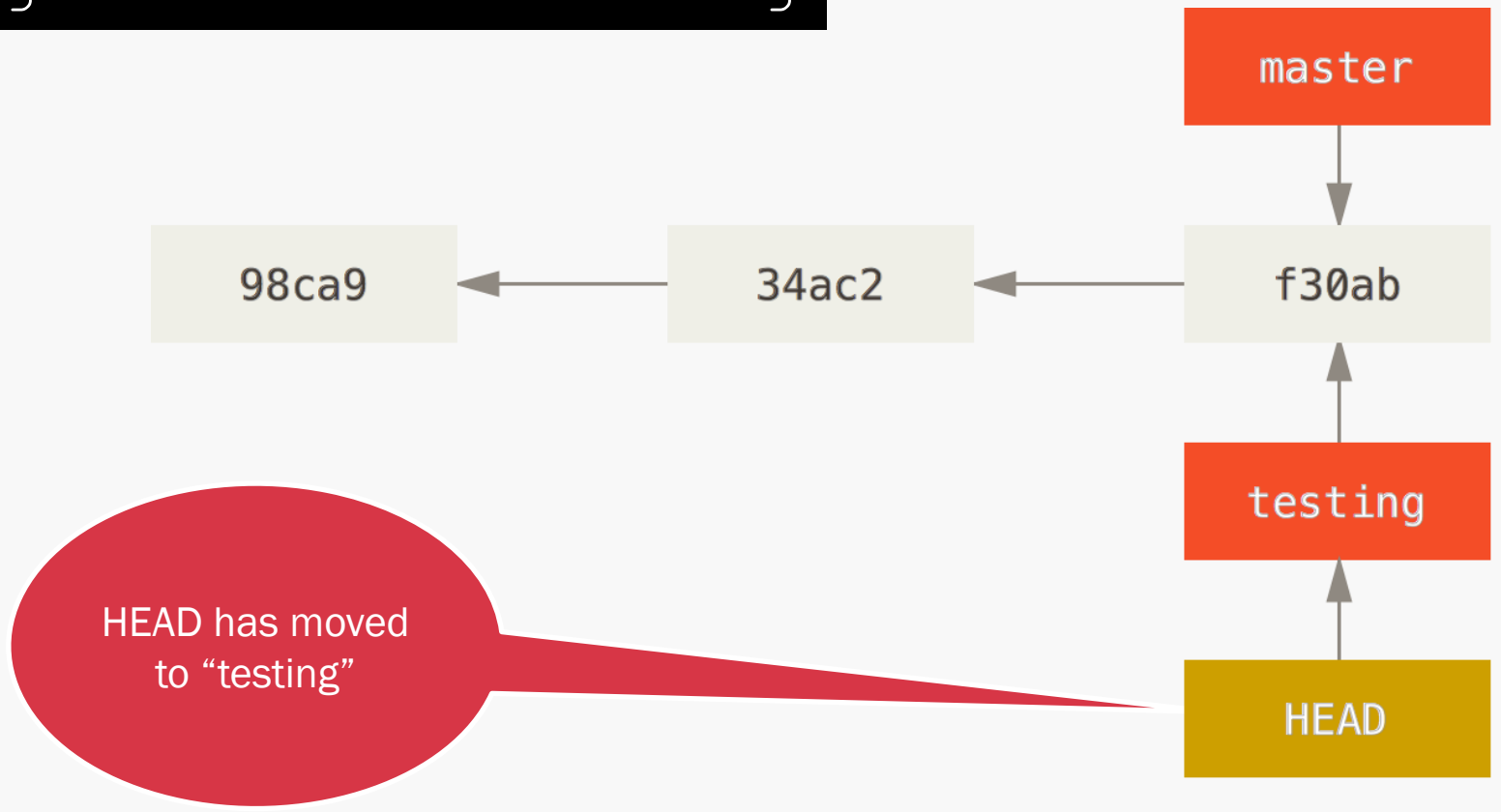
```
$ git branch testing
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Switching branches

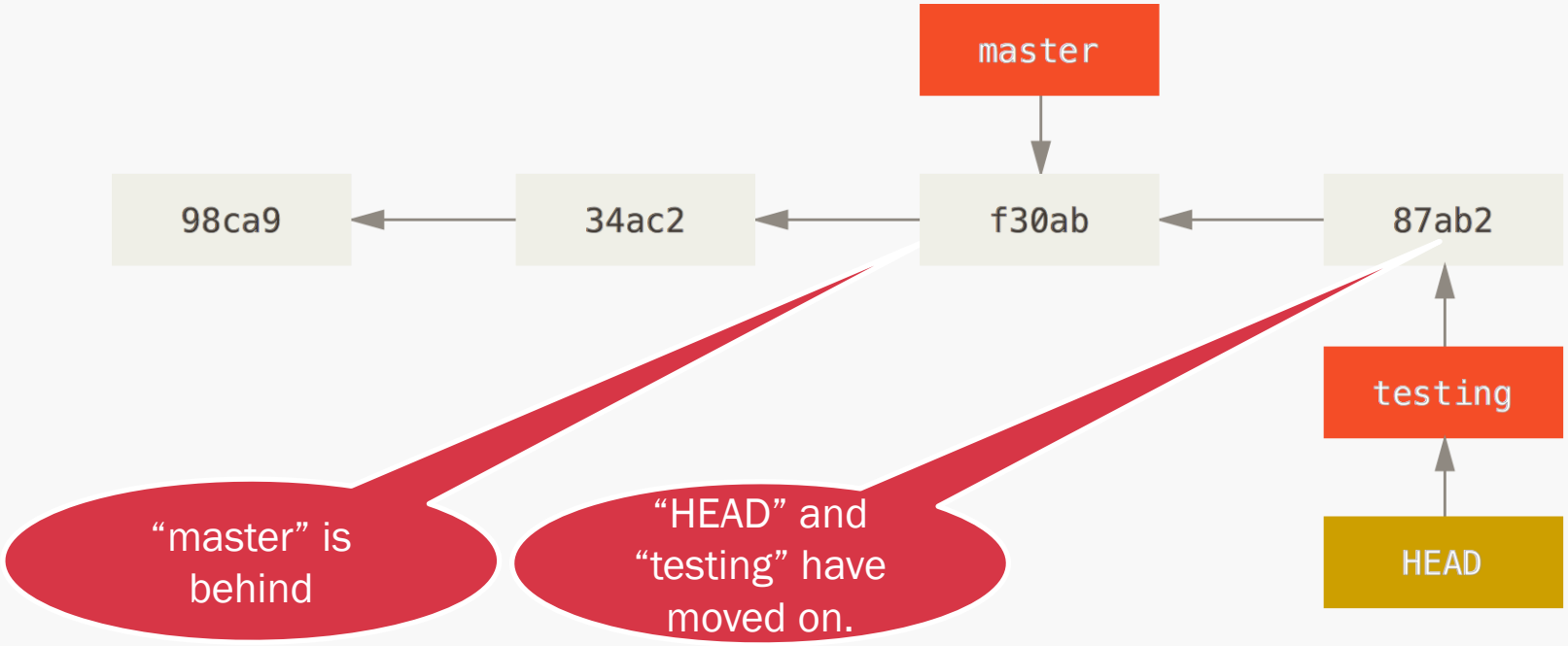
```
$ git checkout testing
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Commit on “testing” branch

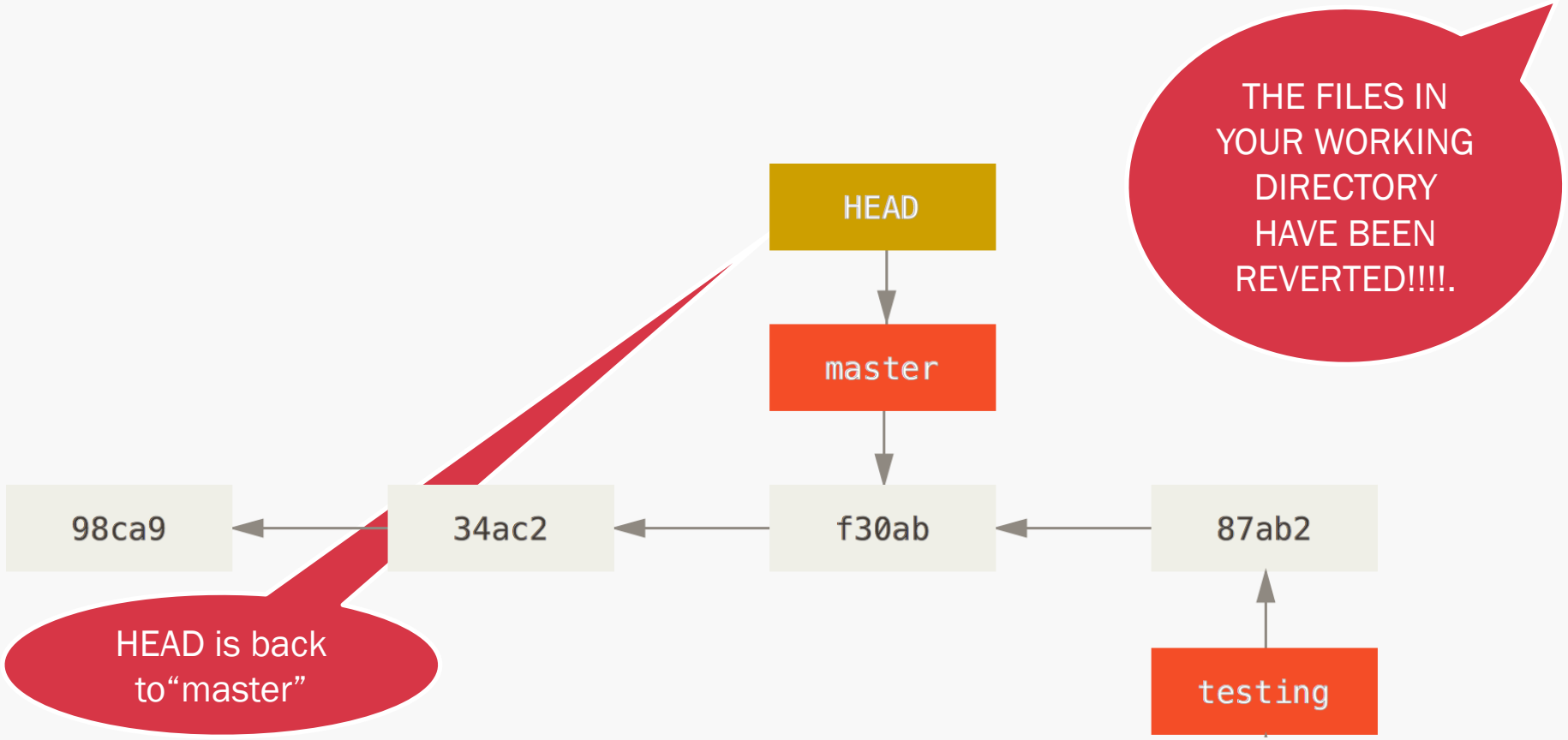
```
$ git commit -a -m “change”
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Go back to “master”

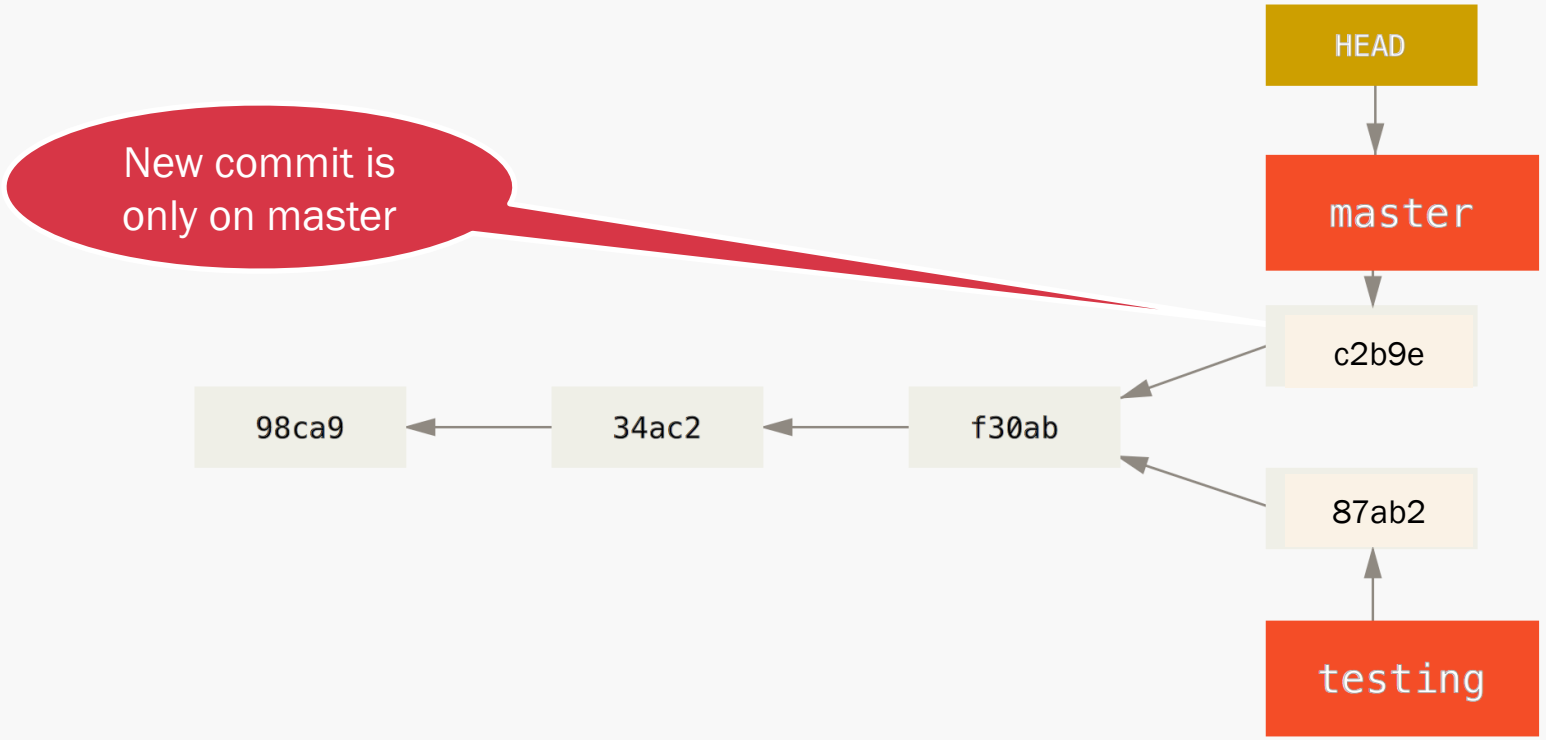
```
$ git checkout master
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

# Now commit again...

```
$ git commit -a -m 'change #2'
```




https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Merging changes

- If branches provide isolation, how do we bring the changes back together?
- We “merge” them.
- The “`git merge <branch>`” command merges the stated branch into the current branch.

# Merge Conflicts

- If the same file has been changed in two different branches, git may not be able to automatically merge the changes.
- At this point, it will indicate a conflict and require you to choose the content you require and commit again.
- You will see content like this.
- Choose and remove the “<“, “>” and “=“.

```
<<<<<<< HEAD
Changes from branch A
=======
Changes from branch B
>>>>>>> feature-branch
```

# Merge vs rebase

- Merges are safe, non-destructive operations that preserve the full commit history
  - This can make the history complex
- You will come across the “rebase” command as you explore git.
- It is risky.
- Never use it on a public branch where public means someone else is working on it.
- See https://www.atlassian.com/git/tutorials/merging-vs-rebasing

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# A common workflow

- Plan a task
- Create a branch for the task
- Do your work on that branch
- Merge the work back to main

- Good practice is to keep the branch short-lived
- To do this, the tasks need to be small

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# User Stories

- A common method for managing requirements is the User Story.
- A user story is a unit of work that brings value to a particular user.
- Think of it as something the user does that benefits from your code.

# User Story Format

- A common format for user stories is:
- As a <role>
  I want <intent>
  so that <benefit achieved>
- Some stories can be very big and are often known as “epics”
- Ideally, we want to work on much smaller tasks.

# Splitting user stories

- There are many tactics for splitting stories:
- SPIDR:
  - Spike
  - Path
  - Interface
  - Data
  - Rule
- User Story Map
  - Narrative flow and steps

# Practice : D'Hondt Method

- There was an election in Wales recently.
- It used the D'Hondt method.
- Wales was split into 16 constituencies and each constituency elected 6 members via D'Hondt.
- There are many variations on the method used across the world.

# How it works

- Count the votes per party.
- Divide each party's votes by the number of seats already won plus one.
- Allocate the seat to the party with the most votes and elect the member from the top of that party's list of candidates for that constituency.
- Repeat until all seats are filled.

# Implement a flexible D'Hondt calculator

- Build in small steps
  - Start with one constituency
  - Start with a fixed number of seats
  - Output parties only
  - Ignore ties
  - Introduce variable size constituencies
  - Introduce candidate lists
  - Introduce multiple constituencies
  - Introduce thresholds
  - …

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Practice using git individually

- Do each step on a branch.
- Implement it.
- Test it (write automated tests ???)
- Merge it.
- Push it.

- Be comfortable with git on your own so that we can grow to use it as a team.

CARDIFF UNIVERSITY
PRIFYSGOL CAERDYDD

# Questions?